GestorPyme

Sistema de Gestión de Clientes, Cotizaciones y Cobranza

Despacho Contable / Consultoría de Servicios Profesionales

DATOS DEL PROYECTO	
Nombre del sistema	GestorPyme v1.0
Tipo de proyecto	Aplicación web SaaS — modo monousuario (un despacho)
Stack tecnológico	SvelteKit · JavaScript · PostgreSQL SUPABASE · Prisma ORM · Clerk (Google OAuth) · Resend · Tailwind CSS
IDE de desarrollo	Cursor / Windsurf con asistencia de IA (Claude Sonnet)
Despliegue	Render.com (Web Service) + SUPABASE (serverless) vercel
Autenticación	Google OAuth 2.0 vía Clerk
Correos transaccionales	Resend API con dominio de prue
Autor	
Matrícula	
Fecha de entrega	3 de julio de 2025 — Sesión 7
Versión del documento	1.0

ARQUITECTURA RESUMIDA	
Frontend	SvelteKit · Svelte Components · Tailwind CSS
Backend / API	SvelteKit API Routes (server endpoints) — sin servidor separado
Base de datos	PostgreSQL en Suupabase

Auth	Clerk · Google OAuth · sesiones manejadas automáticamente por Clerk · hook de protección de rutas en hooks.server.js
Emails	Resend SDK — correos en HTML puro generado con strings de JavaScript
Despliegue	Render.com Web Service conectado al repo GitHub (CI/CD automático en cada push a main)
Variables de entorno	DATABASE_URL · PUBLIC_CLERK_PUBLISHABLE_KEY · CLERK_SECRET_KEY · RESEND_API_KEY · FROM_EMAIL

SECCIÓN 1 — VISIÓN DEL PRODUCTO Y PROBLEMA A RESOLVER

1.1 El problema real

Los despachos contables y consultoras pequeñas en México (1–10 colaboradores) gestionan sus clientes, cotizaciones y cobros usando WhatsApp, hojas de Excel separadas y correo informal. No tienen visibilidad de cuánto les deben, qué cotizaciones están pendientes de aprobar, ni un historial claro de cada cliente. Esto provoca pérdida de ingresos por facturas olvidadas, cotizaciones que nunca se convierten en proyectos, y clientes que se van sin que el despacho sepa por qué.

1.2 La solución propuesta

GestorPyme es una aplicación web privada que permite a los socios y colaboradores de un despacho gestionar su cartera de clientes, emitir cotizaciones profesionales con seguimiento de estado, registrar pagos y visualizar el estado financiero del despacho en un dashboard en tiempo real — todo con login seguro vía Google y notificaciones automáticas por correo electrónico.

1.3 Métricas de éxito del MVP

Métrica 1	El usuario puede registrar un nuevo cliente con todos sus datos en menos de 90 segundos.
Métrica 2	Una cotización puede crearse, enviarse por correo al cliente y marcarse como enviada en menos de 3 minutos.
Métrica 3	El dashboard muestra el total facturado, cobrado y pendiente del mes actual sin que el usuario haga ningún cálculo manual.
Métrica 4	El sistema envía un correo automático al cliente cuando su cotización cambia de estado (enviada → aprobada → facturada).
Métrica 5	El tiempo de carga inicial de cualquier pantalla es menor a 2 segundos en conexión 4G.

1.4 Propuesta de valor diferencial
A diferencia de CRMs genéricos como HubSpot o Salesforce (costosos y sobredimensionados para una PYME), GestorPyme está diseñado específicamente para el flujo de trabajo de un despacho de servicios profesionales mexicano: cotización → aprobación del cliente → entrega del servicio → factura → pago. Sin funciones innecesarias, sin costos de licencia por usuario, desplegado en la nube en minutos.

SECCIÓN 2 — USUARIOS DEL SISTEMA (PERSONAS)	
PERSONA 1	
Nombre / Rol	Carlos Mendoza — Socio Director del Despacho
Descripción	Contador público con 15 años de experiencia. Dirige un despacho de 4 personas en Chihuahua. No programa, pero usa Excel con fluidez y tiene smartphone.
Objetivo principal	Ver de un vistazo cuánto dinero tiene pendiente de cobrar, qué clientes están activos y cuál es el ingreso del mes — sin abrir 5 archivos distintos.
Frustración actual	Pierde cotizaciones porque las hace en Word y las manda por WhatsApp. No sabe cuántas están aprobadas ni cuántas ignoró el cliente.
Frecuencia de uso	Diario — revisa el dashboard cada mañana y registra cobros 2–3 veces por semana.
Nivel técnico	Básico-Intermedio. Usa Google, WhatsApp, Excel. No tolera interfaces complejas.
PERSONA 2	
Nombre / Rol	Daniela Ríos — Asistente Administrativa

Descripción	Administradora de empresas, 26 años. Encargada de capturar clientes, generar cotizaciones y dar seguimiento a pagos. Usuario principal del sistema día a día.
Objetivo principal	Generar cotizaciones rápido, enviarlas por correo directo desde el sistema y saber en qué estado está cada una sin tener que preguntarle a Carlos.
Frustración actual	Tiene que copiar datos del cliente de WhatsApp a Excel, luego a Word para la cotización, luego enviarla por correo. Si el cliente pide cambios, repite todo el proceso.
Frecuencia de uso	Muy alta — usa el sistema 4–6 horas diarias para captura, seguimiento y reportes.
Nivel técnico	Intermedio. Muy cómoda con Google Workspace. Aprende rápido interfaces web.
Dispositivo principal	Laptop Windows en oficina. Raramente móvil para trabajo.
PERSONA 3 (usuario externo — no tiene login)	
Nombre / Rol	Roberto Alvarado — Cliente del despacho
Descripción	Empresario PYME, cliente del despacho. No usa el sistema directamente pero recibe correos automáticos generados por GestorPyme con sus cotizaciones y estados de cuenta.
Objetivo principal	Recibir cotizaciones claras, profesionales, con el desglose de servicios y poder aprobarlas fácilmente.

Frustración actual	Recibe cotizaciones en PDF por WhatsApp, no sabe si el despacho recibió su aprobación, y a veces no recuerda cuánto debe.
Frecuencia de uso	Esporádica — recibe correos cuando el despacho genera o actualiza una cotización. No necesita cuenta.
Nivel técnico	Básico. Solo necesita abrir un correo y leerlo.
Dispositivo principal	Smartphone Android. Los correos deben verse bien en móvil.

SECCIÓN 3 — SUPUESTOS, RESTRICCIONES Y DECISIONES TÉCNICAS	
3.1 Supuestos del proyecto	
Supuesto 1	El despacho tiene una cuenta de Google Workspace o Gmail que usará para el login OAuth. Sin Google, la autenticación no funciona.
Supuesto 2	Se usará el free tier de supabase para PostgreSQL (0.5 GB storage, suficiente para esta etapa). Si el despacho crece, se migra al plan pro.
Supuesto 3	El dominio del despacho estará configurado en Resend para el envío de correos. Sin dominio propio, los correos salen desde onboarding@resend.dev.
Supuesto 4	Render.com free tier tiene cold start de ~30 segundos tras inactividad. El despacho lo acepta para la versión MVP.
Supuesto 5	El sistema es monousuario por despacho (no multi-tenant). Un deploy = un despacho. La escalabilidad multi-tenant es futura.
3.2 Restricciones técnicas confirmadas	
Plataforma	Web (responsive). Mobile-first en vistas de dashboard y clientes. No hay app nativa.
Lenguaje principal	JavaScript en todo el proyecto. Sin TypeScript. Prisma schema es agnóstico al lenguaje..

Framework	SvelteKit con filesystem routing. Rutas en /src/routes/. Vistas en +page.svelte, lógica de servidor en +page.server.js.
ORM / DB	supabase Cero queries SQL en crudo — todo pasa por Prisma Client.
Autenticación	Clerk (Auth.js). Único proveedor: Google OAuth. Sin usuario/contraseña. Sin magic links.
Correos	Resend API. Templates como strings HTML en JavaScript puro. Solo correos transaccionales (no marketing).
UI Components	Tailwind CSS instalado con npx svelte-add@latest tailwindcss. Componentes propios. daisyUI opcional para utilidades rápidas.
Estado del cliente	Svelte stores para estado global ligero. No se usa Redux.
Gráficas	Chart.js con svelte-chartjs (npm install chart.js svelte-chartjs). Integración directa con componentes Svelte.
Deploy	Render.com Web Service. Deploy automático desde rama main de GitHub. Variables de entorno en el panel de Render.
CI/CD	No hay pipeline complejo. Push a main = deploy automático. Tests manuales en esta versión.
Formato monetario	Pesos mexicanos (MXN). Locale es-MX para todos los formatos de fecha y moneda.
3.3 Variables de entorno requeridas (Render + local .env)	

DATABASE_URL	Cadena de conexión PostgreSQL de Neon. Formato: postgresql://user:pass@host/db?sslmode=require
CLERK_SECRET_KEY	Secret Key del backend de Clerk. Se obtiene en clerk.com → Configure → API Keys. Nunca exponerla en el cliente.
PUBLIC_ORIGIN	
PUBLIC_CLERK_PUBLISHABLE_KEY	Publishable Key de Clerk (empieza con pk_test_ en desarrollo). Se obtiene en clerk.com → Configure → API Keys. Esta SÍ puede exponerse en el cliente (prefijo PUBLIC_).
CLERK_SECRET_KEY	
RESEND_API_KEY	API Key generada en resend.com/api-keys. Formato: re_xxxxxxxxxxx
FROM_EMAIL	Correo del remitente verificado en Resend. Ej: noreply@tudespacho.mx

SECCIÓN 4 — HISTORIAS DE USUARIO (BACKLOG COMPLETO)

Formato Scrum / Jira. Cada historia tiene ID único, fórmula 'Como / Quiero / Para que', prioridad, esfuerzo y criterios de aceptación.

Alta — MVP	Media — v1.1	Baja — futuro	XS<1h S 1-2h M 2-4h L 4-8h XL>8h

ID	Como...	...quiero...	...para que...	Prioridad	Esfuerzo	Criterios de Aceptación
ÉPICA 0 — SETUP, DEPLOY Y AUTENTICACIÓN						
US-001	desarrollador / alumno	inicializar el proyecto Next.js 14 con TypeScript, Prisma, shadcn/ui y Clerk con un solo comando de Cursor	pueda tener la base del proyecto lista en menos de 30 minutos sin configuración manual	Alta	M 2-4h	✓ npm create svelte@latest gestorpyme corre sin errores ✓ Prisma conecta a supabase ✓ Tailwind CSS funcionando — una clase bg-indigo-600 aplica correctamente en un +page.svelte de prueba ✓ Clerk configurado, ruta /api/auth/[...nextauth] responde
US-002	desarrollador / alumno	configurar el deploy automático en Render.com conectando el repositorio de GitHub	cada push a main despliegue la app en producción sin intervención manual	Alta	M 2-4h	✓ El Web Service en Render buildea exitosamente (npm run build sin errores) ✓ Variables de entorno configuradas en el panel de Render ✓ La URL pública de Render carga la app correctamente ✓ Un segundo push activa un nuevo deploy automáticamente
US-003	socio del despacho	iniciar sesión en GestorPyme usando mi cuenta de Google	no tenga que recordar otra contraseña y el acceso sea seguro desde el primer día	Alta	S 1-2h	✓ Botón 'Iniciar sesión con Google' visible en la página raíz ✓ Al hacer clic redirige a Google OAuth correctamente ✓ Tras autenticarse, el usuario ve el dashboard y su nombre/foto de Google ✓ Las rutas /dashboard, /clientes, /cotizaciones redirigen a /login si no hay sesión ✓ 'Cerrar sesión' termina la sesión y regresa a /login
ÉPICA 1 — CRUD DE CLIENTES						

US-004	asistente administrativa	ver la lista completa de clientes del despacho con búsqueda por nombre o RFC	encuentre rápido al cliente que necesito sin scrollear toda la lista	Alta	M 2-4h	✓ La ruta /clientes muestra una tabla con: nombre, empresa, RFC, teléfono, correo, fecha de alta ✓ Campo de búsqueda filtra en tiempo real por nombre, empresa o RFC ✓ La tabla pagina de 20 en 20 registros ✓ Carga en menos de 1 segundo con 100 clientes en DB
US-005	asistente administrativa	registrar un nuevo cliente con todos sus datos fiscales y de contacto	el cliente quede en el sistema listo para asignarle cotizaciones desde el momento del alta	Alta	M 2-4h	✓ Formulario con campos: nombre completo, empresa, RFC, correo, teléfono, dirección, notas internas ✓ RFC valida formato mexicano (12 o 13 caracteres alfanuméricos) ✓ Correo valida formato estándar ✓ Al guardar, el cliente aparece inmediatamente en la lista ✓ Se muestra toast de confirmación 'Cliente registrado exitosamente'
US-006	asistente administrativa	editar los datos de un cliente existente cuando cambia su información	el sistema siempre tenga los datos correctos y actualizados	Alta	S 1-2h	✓ Botón 'Editar' en cada fila de la tabla de clientes ✓ Los campos se pre-llenan con los datos actuales del cliente ✓ Al guardar, los cambios se reflejan inmediatamente en la tabla ✓ Se registra en DB la fecha de última modificación
US-007	socio director	ver el perfil completo de un cliente con todo su historial de cotizaciones y pagos	tenga contexto completo antes de una reunión o llamada con ese cliente	Alta	L 4-8h	✓ Ruta /clientes/[id] muestra: datos del cliente + lista de sus cotizaciones + total facturado + total cobrado + saldo pendiente ✓ Las cotizaciones del cliente son clickeables y llevan a /cotizaciones/[id] ✓ Los montos se muestran en formato MXN ($1,234.56) ✓ Si el cliente no tiene cotizaciones, muestra estado vacío con CTA para crear una
US-008	socio director	desactivar un cliente sin borrar su historial cuando deja de ser cliente activo	no perder el registro histórico pero sí poder filtrar por clientes activos solamente	Media	S 1-2h	✓ Botón 'Desactivar' (no eliminar) en el perfil del cliente ✓ Clientes inactivos siguen en DB pero no aparecen en listas por defecto ✓ Toggle 'Mostrar inactivos' en /clientes para verlos cuando se necesite ✓ No se puede crear cotización para un cliente inactivo

ÉPICA 2 — CRUD DE COTIZACIONES						
US-009	asistente administrativa	crear una cotización para un cliente con múltiples conceptos de servicio y precios	el cliente reciba un documento profesional con el desglose claro de lo que se le cobrará	Alta	XL >8h	✓ Formulario en /cotizaciones/nueva: seleccionar cliente, fecha, fecha de vencimiento, número de cotización auto-generado ✓ Sección de conceptos: agregar/quitar filas con (descripción, cantidad, precio unitario, subtotal auto-calculado) ✓ Totales calculados automáticamente: subtotal, IVA 16%, total ✓ Campo de notas adicionales (términos, condiciones) ✓ Al guardar, la cotización queda en estado 'Borrador' ✓ El número de cotización sigue el formato COT-2025-001 (año + consecutivo)
US-010	asistente administrativa	cambiar el estado de una cotización a lo largo de su ciclo de vida	el equipo sepa exactamente en qué paso está cada cotización sin preguntar	Alta	M 2-4h	✓ Estados posibles: Borrador → Enviada → Aprobada → Rechazada → Facturada → Pagada ✓ Botones de transición de estado visibles según el estado actual (no se puede saltar pasos) ✓ Cada cambio de estado registra timestamp y usuario que lo hizo ✓ Al pasar a 'Enviada', se dispara automáticamente el correo al cliente (US-017) ✓ Una cotización 'Rechazada' o 'Pagada' no puede cambiar de estado
US-011	asistente administrativa	buscar y filtrar cotizaciones por cliente, estado o rango de fechas	encuentre rápido la cotización que el cliente me preguntó sin revisar todas	Alta	M 2-4h	✓ /cotizaciones muestra tabla con: número, cliente, fecha, monto total, estado ✓ Filtro por estado (multiselect) ✓ Filtro por cliente (dropdown con búsqueda) ✓ Filtro por rango de fechas (desde / hasta) ✓ Badge de color por estado: azul=borrador, amarillo=enviada, verde=aprobada, rojo=rechazada, morado=facturada, gris=pagada

US-012	asistente administrativa	editar una cotización en estado Borrador antes de enviarla al cliente	pueda corregir errores o ajustar precios sin crear una cotización nueva	Alta	S 1-2h	✓ Solo cotizaciones en estado 'Borrador' tienen botón 'Editar' ✓ Cotizaciones en cualquier otro estado son de solo lectura ✓ Al editar, se puede modificar cualquier campo incluyendo conceptos y precios ✓ Los totales se recalculan automáticamente al cambiar conceptos
US-013	socio director	ver el PDF de una cotización para revisión antes de enviarla o para archivo	tenga el documento final tal como lo verá el cliente	Media	L 4-8h	✓ Botón 'Ver PDF' / 'Descargar PDF' en cada cotización ✓ El PDF incluye: logo del despacho, datos fiscales, datos del cliente, tabla de conceptos, totales, notas ✓ Generado server-side con pdfkit o puppeteer — librerías de Node.js, sin dependencias de framework o puppeteer ✓ El PDF se genera al vuelo (no se almacena en DB)
💰 ÉPICA 3 — CRUD DE PAGOS Y COBRANZA						
US-014	asistente administrativa	registrar un pago parcial o total recibido de un cliente contra una cotización aprobada	el sistema refleje en tiempo real cuánto se ha cobrado y cuánto falta por cobrar	Alta	L 4-8h	✓ Botón 'Registrar pago' disponible en cotizaciones con estado Aprobada o Facturada ✓ Formulario: monto del pago, fecha, método (transferencia / efectivo / cheque / tarjeta), referencia ✓ Una cotización puede tener múltiples pagos parciales ✓ El sistema calcula automáticamente: total pagado, saldo restante ✓ Cuando el saldo restante llega a $0, la cotización cambia automáticamente a estado 'Pagada' ✓ Se muestra historial de pagos en la vista de la cotización

US-015	socio director	ver la lista de cotizaciones con saldo pendiente ordenadas por antigüedad	priorizar el seguimiento de cobro con los clientes que llevan más tiempo sin pagar	Alta	M 2-4h	✓ Vista /cobranza muestra solo cotizaciones Aprobadas o Facturadas con saldo > $0 ✓ Columnas: cliente, número de cotización, monto total, pagado, pendiente, días desde emisión ✓ Ordenadas por días de antigüedad (mayor a menor) por defecto ✓ Color de alerta: amarillo = 15-30 días, rojo = más de 30 días ✓ Total de cartera vencida visible en el encabezado de la vista
US-016	asistente administrativa	eliminar un pago registrado por error antes de que la cotización sea marcada como Pagada	corregir errores de captura sin afectar el historial final	Media	S 1-2h	✓ Cada pago en el historial tiene botón 'Eliminar' (solo si la cotización no está en estado Pagada) ✓ Confirmación modal antes de eliminar ('¿Estás seguro? Esta acción no se puede deshacer') ✓ Al eliminar, los totales de pagado/pendiente se recalculan inmediatamente
✉ ÉPICA 4 — NOTIFICACIONES POR CORREO (RESEND)						
US-017	asistente administrativa	que el sistema envíe automáticamente un correo al cliente cuando su cotización cambia a estado 'Enviada'	el cliente reciba notificación profesional sin que yo tenga que redactar un correo manualmente	Alta	L 4-8h	✓ Al cambiar estado a 'Enviada', se dispara POST a Resend API con template HTML ✓ El correo contiene: saludo personalizado con nombre del cliente, número de cotización, monto total, resumen de conceptos, botón 'Ver cotización' (link al PDF o vista web), datos de contacto del despacho ✓ El correo llega en menos de 60 segundos ✓ Se ve correctamente en Gmail, Outlook y Apple Mail (mobile y desktop) ✓ Si el envío falla, se registra el error y se muestra alerta en el sistema — la cotización sí cambia de estado
US-018	socio director	que el cliente reciba un correo cuando su cotización es aprobada internamente	el cliente sepa que su proyecto está en marcha sin que yo tenga que avisarle por WhatsApp	Media	M 2-4h	✓ Correo disparado al cambiar estado a 'Aprobada' ✓ Template diferente al de 'Enviada': tono más cálido, confirma el inicio del proyecto ✓ Incluye fecha estimada de inicio si está capturada en la cotización ✓ Usa el mismo sistema de Resend que US-017

US-019	socio director	enviar manualmente un recordatorio de pago por correo a un cliente con saldo vencido	hacer seguimiento de cobranza sin salir del sistema ni usar WhatsApp	Media	M 2-4h	✓ Botón 'Enviar recordatorio' en la vista /cobranza por cada cotización con saldo pendiente ✓ Modal de confirmación con preview del correo antes de enviar ✓ El correo incluye: monto pendiente, fecha de la cotización, datos de transferencia del despacho ✓ Se registra en historial de la cotización la fecha/hora del recordatorio enviado ✓ No se puede enviar más de 1 recordatorio por día al mismo cliente
ÉPICA 5 — DASHBOARD Y REPORTES						
US-020	socio director	ver un dashboard con los indicadores financieros clave del despacho al iniciar sesión	tome decisiones informadas en menos de 30 segundos sin abrir ningún reporte adicional	Alta	XL >8h	✓ Tarjetas KPI en la parte superior: Total facturado este mes | Total cobrado este mes | Cartera pendiente | Cotizaciones activas ✓ Gráfica de barras: ingresos cobrados por mes (últimos 6 meses) — con Recharts ✓ Gráfica de dona: distribución de cotizaciones por estado ✓ Lista de las 5 cotizaciones más recientes con su estado ✓ Lista de los 3 clientes con mayor saldo pendiente ✓ Todos los datos se calculan en el servidor (Server Component) — sin fetch en el cliente ✓ El dashboard carga en menos de 2 segundos
US-021	asistente administrativa	ver un resumen de cotizaciones por cliente para el mes actual	saber qué clientes generaron más trabajo y cuánto representa cada uno del total del mes	Media	M 2-4h	✓ Vista /reportes/mes muestra tabla: cliente, # cotizaciones, monto total, monto cobrado, % del total del mes ✓ Ordenable por monto total (mayor a menor) ✓ Selector de mes/año para ver períodos anteriores ✓ Botón exportar a CSV (los datos del mes seleccionado)

SECCIÓN 5 — ALCANCE DEL MVP Y SCHEMA DE BASE DE DATOS

✅ INCLUIDO EN EL MVP — Sesión 7	
✅	Login con Google (Clerk) + protección de rutas
✅	CRUD completo de Clientes (alta, edición, desactivación, perfil con historial)
✅	CRUD completo de Cotizaciones (crear, editar borrador, cambio de estados, vista detalle)
✅	CRUD completo de Pagos (registrar, historial por cotización, eliminación, cálculo automático de saldo)
✅	Vista de Cobranza pendiente con alertas por antigüedad
✅	Envío de correo automático al cliente cuando cotización cambia a estado 'Enviada' (Resend)
✅	Dashboard con 4 KPIs, gráfica de ingresos por mes y gráfica de cotizaciones por estado
✅	Deploy funcional en Render.com con variables de entorno configuradas
✅	Base de datos PostgreSQL en supabase con migraciones Prisma aplicadas
✅	Diseño responsive mobile-first con shadcn/ui y Tailwind CSS

🚫 FUERA DE ALCANCE DEL MVP (versiones futuras)	
🚫	Multi-tenant (múltiples despachos en una instancia) — se diseña para un despacho por deploy
🚫	Generación de PDF de cotizaciones — se entrega en versión 1.1

🚫	Portal de cliente (que el cliente vea sus cotizaciones con login propio) — versión futura
🚫	Facturación electrónica (CFDI / timbrado SAT) — integración con PAC en versión 2.0
🚫	App móvil nativa — la web responsive cubre la necesidad en MVP
🚫	Integración con bancos para conciliación automática de pagos
🚫	Módulo de nómina o gastos del despacho
🚫	Chat interno entre colaboradores

🗄️ SCHEMA DE BASE DE DATOS — Prisma (PostgreSQL)

```
// schema.prisma — GestorPyme MVP

model User {
id String @id @default(cuid())
name String?
email String @unique
emailVerified DateTime?
image String?
createdAt DateTime @default(now())
accounts Account[]
sessions Session[]
}

model Cliente {
id String @id @default(cuid())
nombre String
empresa String?
rfc String? @unique
correo String
telefono String?
direccion String?
```

```
notas String?
activo Boolean @default(true)
creadoEn DateTime @default(now())
actualizadoEn DateTime @updatedAt
cotizaciones Cotizacion[]
}

model Cotizacion {
id String @id @default(cuid())
numero String @unique // COT-2025-001
clienteId String
cliente Cliente @relation(fields: [clienteId], references: [id])
estado EstadoCot @default(BORRADOR)
fecha DateTime @default(now())
vencimiento DateTime?
subtotal Decimal @db.Decimal(12,2)
iva Decimal @db.Decimal(12,2)
total Decimal @db.Decimal(12,2)
notas String?
conceptos Concepto[]
pagos Pago[]
historial HistorialCot[]
creadoEn DateTime @default(now())
actualizadoEn DateTime @updatedAt
}

model Concepto {
id String @id @default(cuid())
cotizacionId String
cotizacion Cotizacion @relation(fields: [cotizacionId], references: [id], onDelete:
Cascade)
descripcion String
cantidad Decimal @db.Decimal(10,2)
precioUnitario Decimal @db.Decimal(12,2)
subtotal Decimal @db.Decimal(12,2)
}

model Pago {
```

```
id String @id @default(cuid())
cotizacionId String
cotizacion Cotizacion @relation(fields: [cotizacionId], references: [id])
monto Decimal @db.Decimal(12,2)
fecha DateTime
metodo MetodoPago
referencia String?
creadoEn DateTime @default(now())
}

model HistorialCot {
id String @id @default(cuid())
cotizacionId String
cotizacion Cotizacion @relation(fields: [cotizacionId], references: [id])
estadoAnterior EstadoCot?
estadoNuevo EstadoCot
nota String?
creadoEn DateTime @default(now())
}

enum EstadoCot {
BORRADOR
ENVIADA
APROBADA
RECHAZADA
FACTURADA
PAGADA
}

enum MetodoPago {
TRANSFERENCIA
EFECTIVO
CHEQUE
TARJETA
}

// Tablas requeridas por Clerk (Account, Session, VerificationToken)
```

SECCIÓN 6 — PROMPT MAESTRO PARA CURSOR / WINDSURF

Copia el prompt de cada celda y pégalo en el chat de Cursor o Windsurf. Sigue el orden numérico — cada prompt depende del anterior.



PROMPT 1 — Setup inicial del proyecto (pegar primero en Cursor)

Inicializa un proyecto SvelteKit con JavaScript (no TypeScript). Cuando el CLI pregunte, elige:
- TypeScript: No
- ESLint: Sí
- Prettier: Sí

Luego instala y configura lo siguiente:

1. PRISMA:
npm install prisma @prisma/client
npx prisma init --datasource-provider postgresql
(Conectar a Supabase mediante el .env)

2. CLERK (autenticación con Google):
npm install @clerk/sveltekit
Configurar en src/hooks.server.js el handle de Clerk
Crear /src/routes/sign-in/+page.svelte con el componente <SignIn />
Proteger rutas privadas con el middleware de Clerk en hooks.server.js

3. TAILWIND CSS:
npx svelte-add@latest tailwindcss

4. RESEND:
npm install resend

Crea el archivo .env con estos placeholders:
DATABASE_URL=""
PUBLIC_CLERK_PUBLISHABLE_KEY=""
CLERK_SECRET_KEY=""
RESEND_API_KEY=""
FROM_EMAIL=""

Crea el schema de Prisma con los modelos: Cliente, Cotizacion, Concepto, Pago, HistorialCot y los enums EstadoCot y MetodoPago (ver hoja " 5️⃣ Alcance y DB Schema").

Ejecuta: npx prisma migrate dev --name init

El layout base en src/routes/+layout.svelte debe tener:
- Sidebar con links a /dashboard, /clientes, /cotizaciones, /cobranza
- Header con nombre y foto del usuario (de Clerk: data.user)
- Botón de cerrar sesión
- Cualquier ruta excepto /sign-in redirige al login si no hay sesión activa (verificar en +layout.server.js con locals.userId de Clerk)



PROMPT 2 — Deploy en Render.com

El proyecto SvelteKit ya corre en local con npm run dev. Necesito desplegarlo en Render.com.

Dame instrucciones exactas para:

1. Crear un Web Service en Render conectado a mi repositorio de GitHub
- Build Command: npm install && npx prisma generate && npm run build
- Start Command: node build/index.js
- Environment: Node

2. Variables de entorno que debo agregar en el panel de Render:
DATABASE_URL, PUBLIC_CLERK_PUBLISHABLE_KEY, CLERK_SECRET_KEY, RESEND_API_KEY, FROM_EMAIL, ORIGIN (URL pública de Render)

3. Cómo ejecutar npx prisma migrate deploy contra Neon en producción
(¿como parte del build command o manualmente desde terminal de Render?)

4. Cómo agregar la URL de producción de Render en el dashboard de Clerk:
- Allowed origins / redirect URLs en clerk.com → Configure → Paths

5. Por qué el adapter de SvelteKit debe ser @sveltejs/adapter-node para Render
(no adapter-auto ni adapter-static)

También explícame cómo crear la base de datos PostgreSQL en supabase y obtener el DATABASE_URL correcto que termina en ?sslmode=require



PROMPT 3 — CRUD de Clientes completo

Implementa el módulo completo de Clientes en src/routes/(dashboard)/clientes/ con SvelteKit y JavaScript:

LISTADO — src/routes/(dashboard)/clientes/+page.server.js + +page.svelte:
- load() hace query a Prisma: prisma.cliente.findMany({ where: { activo: true }, orderBy: { creadoEn: 'desc' } })
- +page.svelte muestra tabla con: nombre, empresa, RFC, correo, teléfono, fecha de alta, botones de acción
- Input de búsqueda que filtra la lista reactivamente con $: clientes filtrados por nombre/RFC/empresa
- Paginación simple de 20 registros con let pagina = 1

CREAR CLIENTE — Form Action 'crear' en +page.server.js:
- Campos del formulario en modal (boolean showModal con {#if showModal}): nombre (requerido), empresa, RFC (validar 12-13 caracteres), correo (requerido), teléfono, dirección, notas
- Validación con Zod en la action del servidor ANTES de tocar Prisma
- Si hay errores de validación: return fail(400, { errors, values }) para mostrarlos en el form
- Si éxito: invalidate('app:clientes') + mensaje de confirmación con una variable flash

EDITAR CLIENTE — Form Action 'editar':
- Misma lógica que crear pero con los datos pre-cargados en el formulario
- Solo habilitado si el cliente está activo

PERFIL — src/routes/(dashboard)/clientes/[id]/+page.server.js:
- load() trae: cliente con sus cotizaciones, calcula total facturado, cobrado y pendiente
- +page.svelte muestra: datos del cliente, tabla de cotizaciones, 3 KPIs de montos
- Botón "Desactivar cliente" con Form Action 'desactivar' + confirm() antes de enviar

Todo en JavaScript puro. Sin TypeScript.



PROMPT 4 — CRUD de Cotizaciones con estados

Implementa el módulo completo de Cotizaciones en src/routes/(dashboard)/cotizaciones/ con SvelteKit:

LISTADO — +page.server.js + +page.svelte:
- load() trae cotizaciones con su cliente relacionado (include: { cliente: true })
- Tabla: número, cliente, fecha, vencimiento, total, estado con badge de color
- Filtros reactivos en Svelte: $: filtradas = cotizaciones.filter(c => ...) por estado y cliente
- Colores de badge por estado: BORRADOR=gris, ENVIADA=amarillo, APROBADA=verde, RECHAZADA=rojo, FACTURADA=morado, PAGADA=teal

CREAR COTIZACIÓN — /cotizaciones/nueva/+page.svelte + +page.server.js:
- load() calcula el número siguiente (COT-2025-001) consultando el último en DB
- Formulario: selector de cliente, fechas, tabla de conceptos dinámica
- Tabla de conceptos: array en Svelte let conceptos = []. Botón "Agregar concepto" hace conceptos = [...conceptos, {descripcion:'', cantidad:1, precio:0}]
- Subtotal de cada fila y totales (subtotal, IVA 16%, total) calculados con $: reactivo
- Form Action 'guardar': valida con Zod, crea Cotizacion + Conceptos en una transacción Prisma
- Dos botones de submit: "Guardar borrador" (estado BORRADOR) y "Enviar al cliente" (estado ENVIADA + dispara correo)

VISTA DETALLE — /cotizaciones/[id]/+page.svelte:
- load() trae cotización completa: conceptos, pagos, historial de estados, cliente
- Vista solo-lectura con todos los datos
- Historial de cambios de estado con timestamps
- Sección de pagos: lista de pagos registrados + resumen (pagado / pendiente)
- Botón 'Registrar pago' abre modal {#if showPago}
- Botones de transición de estado según el estado actual (solo los permitidos)

CAMBIO DE ESTADO — Form Action 'cambiarEstado':
- Valida que la transición sea permitida (ej: BORRADOR → ENVIADA está bien, PAGADA → BORRADOR no)
- Actualiza estado en Prisma
- Crea registro en HistorialCot con timestamp
- Si nuevo estado es ENVIADA: llama a la función de envío de correo con Resend
- return { success: true } o fail(400, { error })

Todo en JavaScript. Form Actions de SvelteKit — sin fetch manual al servidor.



PROMPT 5 — Pagos y vista de Cobranza

Implementa el módulo de Pagos y la vista de Cobranza en SvelteKit:

REGISTRAR PAGO (modal dentro de /cotizaciones/[id]/+page.svelte):
- let showPago = false. Modal con {#if showPago}
- Formulario: monto (number, formato MXN), fecha (date), método (select: TRANSFERENCIA/EFECTIVO/CHEQUE/TARJETA), referencia
- Form Action 'registrarPago' en +page.server.js:
* Crea el Pago en DB
* Calcula nueva suma de pagos: SELECT SUM(monto) FROM Pago WHERE cotizacionId = id
* Si suma >= total de cotización: actualizar estado a PAGADA + crear HistorialCot
* return { success: true, saldoPendiente: nuevoSaldo }

HISTORIAL DE PAGOS (en la misma vista de detalle):
- Lista de pagos con: fecha, monto formateado en MXN, método, referencia, botón Eliminar
- Resumen al final: Total | Pagado | Pendiente (colorear pendiente en verde/amarillo/rojo)
- Form Action 'eliminarPago':
* Solo ejecuta si cotización NO está en estado PAGADA
* Borra el pago
* Recalcula y muestra nuevo saldo
* Modal de confirmación antes: {#if confirmDelete}

VISTA DE COBRANZA — /cobranza/+page.server.js + +page.svelte:
- load() query Prisma:
prisma.cotizacion.findMany({
where: { estado: { in: ['APROBADA', 'FACTURADA'] } },
include: { cliente: true, pagos: true }
})
Calcular en JS: saldoPendiente = cotizacion.total - suma(cotizacion.pagos)
Filtrar: solo donde saldoPendiente > 0
Calcular diasTranscurridos = diferencia en días desde cotizacion.fecha hasta hoy

- +page.svelte tabla: cliente, número, fecha, total, pagado, PENDIENTE, días
- Color de fila con class: {diasTranscurridos > 30 ? 'bg-red-50' : diasTranscurridos > 15 ? 'bg-yellow-50' : ''}
- Header: "Cartera pendiente total: $XX,XXX.XX MXN" (suma de todos los saldos)

- Ordenar por días transcurridos (mayor primero): .sort((a,b) => b.dias - a.dias)
- Botón "Enviar recordatorio" por fila → Form Action 'enviarRecordatorio' que llama a Resend



PROMPT 6 — Correos con Resend (HTML puro en JavaScript)

Implementa el sistema completo de correos transaccionales con Resend en SvelteKit.

——— PASO 1: CUENTA Y CONFIGURACIÓN EN RESEND ———
1. Crear cuenta gratuita en resend.com (free tier: 3,000 correos/mes, 100/día)
2. En el dashboard de Resend → "API Keys" → crear nueva API Key → copiar el valor (empieza con re_)
3. En el dashboard de Resend → "Domains" → agregar el dominio del despacho (ej: tudespacho.mx)
- Resend mostrará registros DNS (TXT, MX, DKIM) que hay que agregar en el proveedor del dominio
- Si no se tiene dominio propio: usar onboarding@resend.dev como remitente (solo para pruebas)
4. Agregar al archivo .env:
RESEND_API_KEY="re_xxxxxxxxxxxxxxxxxxxx"
FROM_EMAIL="noreply@tudespacho.mx"

——— PASO 2: INSTALAR RESEND ———
npm install resend

——— PASO 3: CLIENTE DE RESEND — src/lib/email.js ———
```
import { Resend } from 'resend'
import { RESEND_API_KEY, FROM_EMAIL } from '$env/static/private'

const resend = new Resend(RESEND_API_KEY)

export async function sendEmail({ to, subject, html }) {
try {
const { data, error } = await resend.emails.send({
from: FROM_EMAIL,
to: [to],
subject,
html
})
if (error) {
console.error('[Resend] Error al enviar:', error)
return { ok: false, error }
```

```
}
console.log('[Resend] Correo enviado. ID:', data.id)
return { ok: true, id: data.id }
} catch (err) {
console.error('[Resend] Excepción:', err)
return { ok: false, error: err.message }
}
}

──── PASO 4: TEMPLATES HTML — src/lib/emailTemplates.js ────
(HTML puro en strings de JavaScript — sin Svelte, sin dependencias extra)

export function templateCotizacionEnviada({ cliente, cotizacion, conceptos }) {
const formatMXN = (n) => new Intl.NumberFormat('es-MX', { style: 'currency',
currency: 'MXN' }).format(n)

const filasConceptos = conceptos.map(c => `
<tr>
<td style="padding:8px 12px;border-bottom:1px solid
#e5e7eb;">${c.descripcion}</td>
<td style="padding:8px 12px;border-bottom:1px solid #e5e7eb;text-
align:center;">${c.cantidad}</td>
<td style="padding:8px 12px;border-bottom:1px solid #e5e7eb;text-
align:right;">${formatMXN(c.precioUnitario)}</td>
<td style="padding:8px 12px;border-bottom:1px solid #e5e7eb;text-
align:right;">${formatMXN(c.subtotal)}</td>
</tr>`).join('')

return `<!DOCTYPE html>
<html lang="es">
<body style="font-family:Arial,sans-
serif;background:#f9fafb;margin:0;padding:20px;">
<div style="max-width:600px;margin:0 auto;background:#ffffff;border-
radius:8px;overflow:hidden;box-shadow:0 1px 3px rgba(0,0,0,0.1);">
<div style="background:#4F46E5;padding:24px 32px;">
<h1 style="color:#ffffff;margin:0;font-size:22px;">Cotización
${cotizacion.numero}</h1>
</div>
```

```
<div style="padding:32px;">
<p style="color:#374151;font-size:15px;">Estimado/a
<strong>${cliente.nombre}</strong>,</p>
<p style="color:#374151;font-size:15px;">Le enviamos la cotización
<strong>${cotizacion.numero}</strong> con el siguiente desglose de servicios:</p>
<table style="width:100%;border-collapse:collapse;margin:20px 0;font-size:14px;">
<thead>
<tr style="background:#f3f4f6;">
<th style="padding:10px 12px;text-align:left;color:#6b7280;">Descripción</th>
<th style="padding:10px 12px;text-align:center;color:#6b7280;">Cant.</th>
<th style="padding:10px 12px;text-align:right;color:#6b7280;">Precio Unit.</th>
<th style="padding:10px 12px;text-align:right;color:#6b7280;">Subtotal</th>
</tr>
</thead>
<tbody>${filasConceptos}</tbody>
</table>
<div style="text-align:right;margin-
top:16px;padding:16px;background:#eef2ff;border-radius:6px;">
<p style="margin:4px 0;color:#6b7280;font-size:13px;">Subtotal:
${formatMXN(cotizacion.subtotal)}</p>
<p style="margin:4px 0;color:#6b7280;font-size:13px;">IVA (16%):
${formatMXN(cotizacion.iva)}</p>
<p style="margin:0;color:#4F46E5;font-size:20px;font-weight:bold;">Total:
${formatMXN(cotizacion.total)}</p>
</div>
${cotizacion.vencimiento ? `<p style="color:#6b7280;font-size:13px;margin-
top:16px;">Esta cotización tiene vigencia hasta el <strong>${new
Date(cotizacion.vencimiento).toLocaleDateString('es-MX')}</strong>.</p>` : ''}
${cotizacion.notas ? `<div style="background:#f9fafb;border-left:3px solid
#4F46E5;padding:12px 16px;margin-top:16px;"><p style="margin:0;font-
size:13px;color:#374151;">${cotizacion.notas}</p></div>` : ''}
</div>
<div style="background:#f3f4f6;padding:20px 32px;font-
size:12px;color:#9ca3af;text-align:center;">
<p style="margin:0;">Para cualquier duda, contáctenos al correo o teléfono del
despacho.</p>
</div>
</div>
```

```
</body>
</html>`
}

export function templateRecordatorioPago({ cliente, cotizacion, saldoPendiente }) {
const formatMXN = (n) => new Intl.NumberFormat('es-MX', { style: 'currency', currency: 'MXN' }).format(n)
return `<!DOCTYPE html>
<html lang="es">
<body style="font-family:Arial,sans-serif;background:#f9fafb;margin:0;padding:20px;">
<div style="max-width:600px;margin:0 auto;background:#ffffff;border-radius:8px;overflow:hidden;">
<div style="background:#1F2937;padding:24px 32px;">
<h1 style="color:#ffffff;margin:0;font-size:20px;">Recordatorio de pago</h1>
</div>
<div style="padding:32px;">
<p style="color:#374151;">Estimado/a <strong>${cliente.nombre}</strong>,</p>
<p style="color:#374151;">Le recordamos que tiene un saldo pendiente correspondiente a la cotización <strong>${cotizacion.numero}</strong>.</p>
<div style="text-align:center;padding:24px;background:#fef3c7;border-radius:8px;margin:20px 0;">
<p style="margin:0;color:#92400e;font-size:13px;">Saldo pendiente</p>
<p style="margin:8px 0 0;color:#92400e;font-size:28px;font-weight:bold;">${formatMXN(saldoPendiente)}</p>
</div>
<p style="color:#374151;font-size:14px;">Para cualquier aclaración, no dude en contactarnos. Agradecemos su preferencia.</p>
</div>
</div>
</body>
</html>`
}

——— PASO 5: INTEGRACIÓN EN FORM ACTIONS ———
En la Form Action 'cambiarEstado' de /cotizaciones/[id]/+page.server.js,
cuando nuevoEstado === 'ENVIADA':
```

```
import { sendEmail } from '$lib/email.js'
import { templateCotizacionEnviada } from '$lib/emailTemplates.js'

// ... después de actualizar el estado en Prisma:
const html = templateCotizacionEnviada({ cliente, cotizacion, conceptos })
const { ok, error } = await sendEmail({
to: cliente.correo,
subject: `Cotización ${cotizacion.numero} — ${cotizacion.total} MXN`,
html
})

// Registrar resultado en historial sin importar si hubo error:
await prisma.historialCot.create({
data: {
cotizacionId: cotizacion.id,
estadoAnterior: estadoAnterior,
estadoNuevo: 'ENVIADA',
nota: ok ? `Correo enviado a ${cliente.correo}` : `Error al enviar correo: ${error}`
}
})

──── PASO 6: PROBAR EN LOCAL ────
Para probar sin dominio real, usar la dirección de prueba de Resend:
- FROM_EMAIL="onboarding@resend.dev"
- El correo solo llega a la cuenta de Resend con la que te registraste (en el free tier sin dominio)
- Una vez configurado el dominio, los correos llegan a cualquier destinatario
```



PROMPT 7 — Dashboard con Recharts

Implementa el Dashboard en /src/routes/(dashboard)/dashboard/+page.server.js y +page.svelte:

LOAD FUNCTION (todo en el servidor, cero fetch en el cliente):

```
export const load = async ({ locals }) => {
const ahora = new Date()
const inicioMes = new Date(ahora.getFullYear(), ahora.getMonth(), 1)

// KPI 1: Total facturado este mes
const cotizacionesMes = await prisma.cotizacion.findMany({
where: { estado: { not: 'BORRADOR' }, creadoEn: { gte: inicioMes } }
})
const totalFacturado = cotizacionesMes.reduce((s, c) => s + Number(c.total), 0)

// KPI 2: Total cobrado este mes
const pagosMes = await prisma.pago.findMany({ where: { fecha: { gte: inicioMes } } })
const totalCobrado = pagosMes.reduce((s, p) => s + Number(p.monto), 0)

// KPI 3: Cartera pendiente total
const cotsPendientes = await prisma.cotizacion.findMany({
where: { estado: { in: ['APROBADA', 'FACTURADA'] } },
include: { pagos: true }
})
const carteraPendiente = cotsPendientes.reduce((s, c) => {
const pagado = c.pagos.reduce((sp, p) => sp + Number(p.monto), 0)
return s + (Number(c.total) - pagado)
}, 0)

// KPI 4: Cotizaciones activas
const cotsActivas = await prisma.cotizacion.count({
where: { estado: { in: ['ENVIADA', 'APROBADA', 'FACTURADA'] } }
})

// Gráfica 1: Ingresos últimos 6 meses (calcular en JS agrupando pagos por mes)
// Gráfica 2: Cotizaciones por estado (prisma.cotizacion.groupBy estado)
// Lista: últimas 5 cotizaciones
// Lista: top 3 clientes con mayor pendiente
```

```
return { totalFacturado, totalCobrado, carteraPendiente, cotsActivas,
ingresosPorMes, cotsPorEstado, ultimasCots, topClientes }
}

+PAGE.SVELTE:
- 4 tarjetas KPI con íconos (usar emojis o SVG simples — sin librería de íconos)
- Gráfica de barras con Chart.js via svelte-chartjs:
<script>import { Bar } from 'svelte-chartjs'</script>
<Bar data={chartData} options={chartOptions} />
- Gráfica de dona (Doughnut) para cotizaciones por estado
- Tablas de últimas cotizaciones y top clientes con saldo pendiente
- Todos los montos formateados con:
new Intl.NumberFormat('es-MX', { style: 'currency', currency: 'MXN' }).format(monto)
```